

PLAN DE ARQUITECTURA EXHAUSTIVO

App desktop financiera local para Gmail, Mercado Pago, archivos bancarios y Money Manager Ex (MMEX)

Documento técnico para Ricardo

Versión 1.0 - 02 de mayo de 2026

Veredicto arquitectónico

La app desktop no debe ser un lector de correos que escribe directamente en MMEX. Debe ser una plataforma local de conciliación financiera, con base canónica propia, conectores por fuente, revisión humana controlada y exportación segura hacia Money Manager Ex.

Campo	Descripción
Cliente / usuario	Ricardo
Objetivo	Construir una app desktop local para capturar, normalizar, deduplicar, revisar, analizar y exportar movimientos financieros familiares hacia MMEX.
Restricción principal	Privacidad local: no enviar documentos bancarios ni datos sensibles a servicios externos.
Estrategia recomendada	Arquitectura local-first con staging SQLite, workers de extracción, motor de conciliación y exportación MMEX por CSV/QIF en MVP.
Fuentes base	Informes técnicos: Claude, Gemini, NotebookLM y OpenAI; más las decisiones de arquitectura definidas durante la conversación.

Índice

1. Resumen ejecutivo
2. Objetivo funcional de la app
3. Principios rectores de arquitectura
4. Arquitectura general
5. Stack tecnológico recomendado
6. Capas y módulos del sistema
7. Fuentes de datos e ingesta
8. Capa RAW y conservación de evidencia
9. Modelo de datos local
10. Normalización, categorización y reglas
11. Dedupe y conciliación
12. Integración con Money Manager Ex
13. Seguridad, privacidad y backups
14. Dashboard financiero
15. Scheduler y automatización local
16. Estructura del proyecto
17. Contratos JSON entre C# y Python
18. Validaciones obligatorias
19. Roadmap de desarrollo
20. Estrategia de pruebas
21. Riesgos y mitigaciones
22. Criterios de éxito del MVP
23. Apéndices técnicos

1. Resumen ejecutivo

Este documento propone una arquitectura completa para desarrollar una app desktop financiera local, orientada a consolidar movimientos provenientes de Gmail, Mercado Pago, archivos bancarios, estados de cuenta, PDFs, imágenes y entradas manuales, para luego analizarlos y exportarlos a Money Manager Ex (MMEX).

La decisión central es separar la captura y transformación de datos del libro contable final. En otras palabras: MMEX debe ser el destino contable, no la base operativa de ingestión. La app debe mantener una base canónica propia en SQLite, con evidencia cruda, normalización, deduplicación, revisión humana y exportación controlada.

Decisión principal

No se recomienda escribir directamente sobre el archivo .mmb en la primera versión. El MVP debe exportar CSV/QIF compatible con MMEX. La escritura directa SQLite debe quedar para una fase avanzada, con backups, validación de esquema y transacciones ACID.

- Los correos son una fuente rápida, pero incompleta: no todas las transacciones llegan por email.
- Los estados de cuenta CSV/OFX/QIF/PDF son mejores como fuente de verdad periódica, pero llegan tarde y pueden duplicar correos previos.
- La app debe fusionar movimientos de distintas fuentes en una sola transacción canónica.
- Los pagos rechazados deben alimentar alertas, pero no afectar el balance contable.
- Los pagos de tarjetas deben modelarse como transferencias internas cuando la tarjeta existe como cuenta MMEX.
- El sistema debe operar localmente, preservando evidencia y evitando enviar datos bancarios a la nube.

2. Objetivo funcional de la app

La app debe funcionar como una consola local de control financiero familiar. No debe limitarse a importar datos; debe ayudar a interpretar, depurar y conciliar movimientos antes de que entren a MMEX.

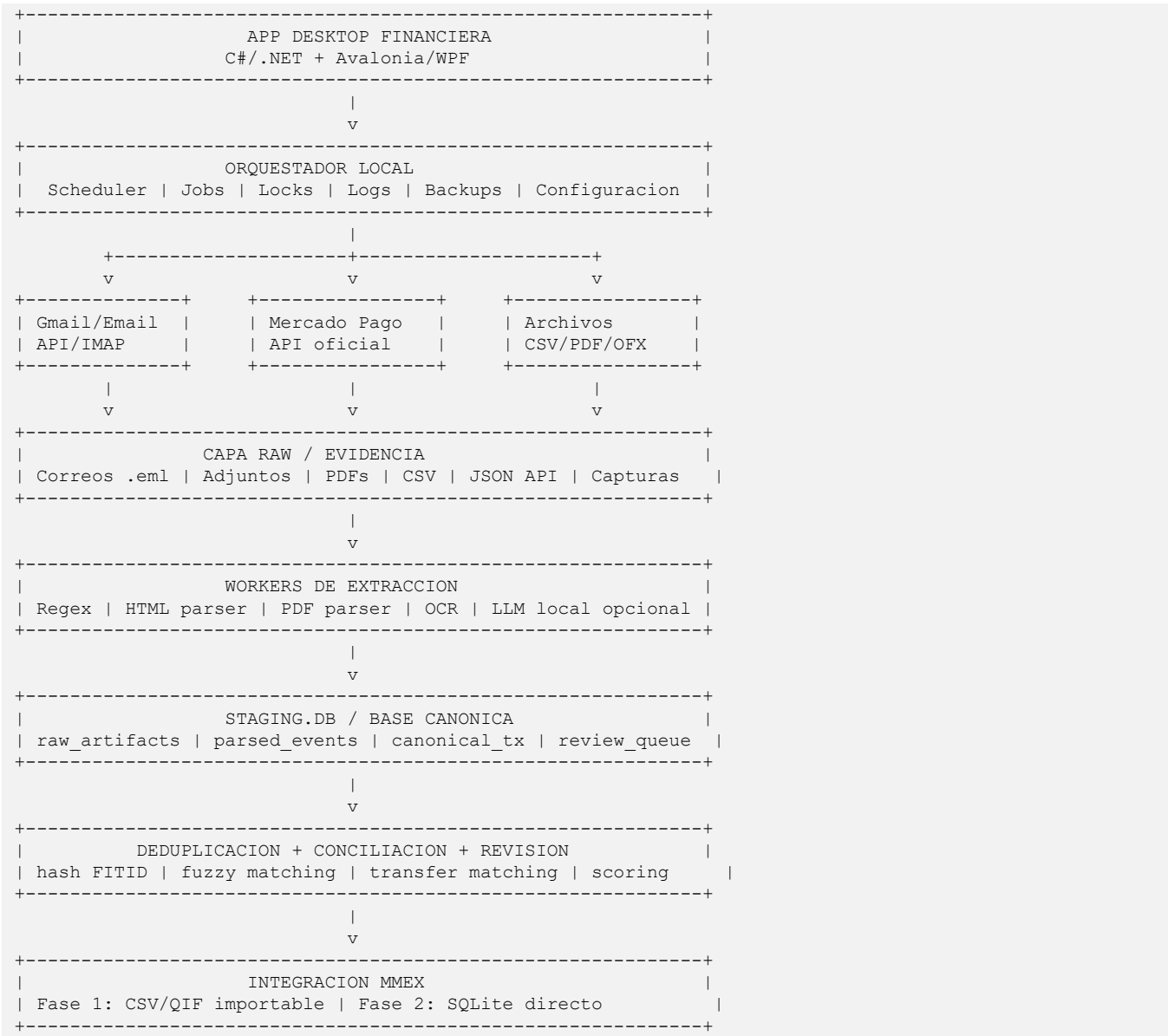
Objetivo	Descripción operativa
Capturar movimientos	Leer Gmail/correo, API de Mercado Pago, carpetas locales, CSV, OFX, QIF, PDF, imágenes y entradas manuales.
Detectar eventos financieros	Compras, pagos, transferencias, abonos, reembolsos, cuotas, pagos rechazados y suscripciones.
Normalizar datos	Convertir fechas, montos, comercios, cuentas, tarjetas y categorías a un modelo canónico estable.
Evitar duplicados	Fusionar correo + estado de cuenta + PDF + API cuando representan el mismo movimiento.
Revisar casos dudosos	Enviar a cola de revisión humana los movimientos con baja confianza, balances inconsistentes o cuenta desconocida.
Exportar a MMEX	Generar CSV/QIF importable en el MVP y habilitar escritura directa SQLite en fase avanzada.
Analizar finanzas	Mostrar dashboard de gastos, ingresos, suscripciones, pagos rechazados, deudas y movimientos pendientes.

3. Principios rectores de arquitectura

Principio	Decisión arquitectónica
Local-first	Toda la persistencia operativa vive en SQLite local y en carpetas controladas por el usuario.
MMEX como destino	El archivo .mmb/.emb no debe ser usado como staging ni como punto de ingestión inicial.
Staging obligatorio	Todo artefacto se procesa primero en staging.db antes de exportarse.
Evidencia preservada	Correos, PDFs, CSV, JSON API, OCR y extractos originales se conservan con hash SHA-256.
Idempotencia	Reprocesar un lote no debe duplicar transacciones.
Revisión humana	El sistema automatiza lo repetible y deriva lo ambiguo a revisión.
Seguridad por defecto	Tokens cifrados, logs sin secretos, backups automáticos y mínimo privilegio.
Automatización progresiva	Primero importación segura por CSV/QIF; después, SQLite directo si el flujo es estable.

4. Arquitectura general

La arquitectura propuesta se organiza en capas: UI desktop, orquestador local, conectores, capa RAW, workers de extracción, base canónica, motor ETL, deduplicación/conciliación, revisión humana e integración MMEX.



5. Stack tecnológico recomendado

La recomendación más sólida es una arquitectura híbrida: C#/.NET para la app desktop, dominio, UI, seguridad y orquestación; Python para extracción avanzada, parsing, OCR, normalización documental y fuzzy matching.

Capa	Tecnología recomendada	Motivo
App desktop	C# .NET 8/9 + Avalonia UI o WPF	Interfaz robusta, buen soporte en Windows y empaquetado profesional.
Dominio y reglas	C#/.NET	Mantiene reglas de negocio, estados, validaciones y seguridad en un núcleo estable.
Base local	SQLite + WAL	Simple, portable, local, sin servidor y suficiente para el volumen doméstico.
Secretos	Windows DPAPI / Credential Manager	Evita guardar tokens OAuth o credenciales en texto plano.

Parsing PDF/OCR	Python + pdfplumber + PyMuPDF + Tesseract	Ecosistema superior para documentos bancarios y OCR local.
Normalización	Python + pandas + RapidFuzz	Muy efectivo para fechas, montos, CSV heterogéneos y comercios difusos.
IA local opcional	Ollama/Qwen local	Ayuda ante plantillas de correo cambiantes sin enviar datos a la nube.
MMEX	CSV/QIF primero; SQLite directo después	Reduce riesgo de corrupción y permite madurez progresiva.

Recomendación objetiva

Hacerlo todo en C# complicaría PDF/OCR/parsing. Hacerlo todo en Python complicaría la experiencia desktop y el empaquetado. La solución híbrida es menos purista, pero más robusta.

6. Capas y módulos del sistema

Módulo	Responsabilidad
Desktop UI	Dashboard, bandejas, revisión, reglas, configuración, exportaciones y reportes.
Orquestador local	Ejecutar jobs, controlar locks, programar sincronizaciones, crear lotes y registrar logs.
Connectors	Gmail, IMAP opcional, Mercado Pago, carpetas locales, entrada manual y futuro browser asistido.
File Storage	Guardar artefactos crudos, adjuntos, hashes, OCR y versiones procesadas.
Extraction Workers	Convertir emails, HTML, CSV, PDF, OCR e imágenes en candidatos JSON.
ETL Engine	Normalizar fechas, montos, dirección, cuentas, payees, categorías y tipos de movimiento.
Dedupe Engine	Detectar duplicados exactos, hashes de negocio, fuzzy matches y ventanas temporales.
Reconciliation Engine	Fusionar fuentes, detectar transferencias internas, pagos de tarjeta y reembolsos.
Review Queue	Gestionar casos dudosos, errores de parser y validaciones fallidas.
MMEX Exporter	Generar CSV/QIF y, en fase avanzada, insertar en SQLite con transacciones ACID.
Security Layer	Cifrado de secretos, políticas de logs, backups, borrado seguro y permisos.

7. Fuentes de datos e ingesta

La app no debe asumir una única fuente de verdad. Cada fuente debe tener un conector, un parser, reglas de confianza y un método de conciliación posterior.

Fuente	Uso	Confiabilidad	Tratamiento
Gmail/correo	Captura temprana de compras, pagos, transferencias y rechazos.	Media	Parser HTML/texto, reglas por banco, evidencia .eml y dedupe posterior.
Mercado Pago API	Movimientos estructurados de wallet, pagos, cobros, comisiones y reembolsos.	Alta	OAuth oficial, JSON API, normalización directa.
CSV bancario	Cartolas y estados de cuenta estructurados.	Alta	Adaptador por banco, validación de saldo y dedupe contra emails.
OFX/QIF	Formatos financieros de intercambio.	Alta	Parser estructurado y exportación/importación más limpia.
PDF nativo	Estados de cuenta o comprobantes con texto embebido.	Media	pdfplumber/PyMuPDF, extracción tabular y revisión.
PDF escaneado/imagen	Capturas, boletas o cartolas sin texto.	Baja	OCR local, confianza baja y revisión humana.
Entrada manual	Movimientos que nunca llegan por correo ni exporte.	Variable	Formulario local con validaciones y evidencia opcional.
Navegación asistida	Descarga manual guiada desde bancos complejos.	Media	Browser visible, sin evasión de controles ni scraping invisible.

7.1. Gmail y correo financiero

- Conectar con Gmail mediante OAuth; IMAP puede quedar como alternativa.

- Filtrar correos por remitente, asunto, etiquetas y palabras clave financieras.
- Considerar dos destinatarios: el correo principal y el correo redirigido.
- Guardar el mensaje bruto, texto limpio, HTML, adjuntos y hashes.
- Extraer candidatos, no transacciones definitivas.
- Marcar pagos rechazados como eventos informativos que no afectan balance.

Consulta Gmail sugerida:
 from:(bancoestado OR falabella OR cmr OR bci OR mach OR mercadopago)
 OR subject:(compra OR cargo OR transferencia OR abono OR rechazado OR pago)
 OR ("pago rechazado" OR "compra realizada" OR "transferencia recibida")

7.2. Mercado Pago

- Usar API oficial con OAuth, no scraping.
- Importar pagos realizados, pagos recibidos, reembolsos, comisiones y transferencias a banco.
- Normalizar wallet como una cuenta propia en MMEX.
- Deducir comisiones como gasto financiero cuando corresponda.
- Conciliar correos de Mercado Pago con movimientos API para no duplicar.

7.3. Bancos chilenos y scraping

Para BancoEstado, Falabella/CMR, Mach/BCI y fuentes similares, la app no debe depender de scraping bancario desatendido. La estrategia recomendable es correo + exportes + descarga manual guiada + procesamiento local. Si se implementa browser asistido, debe ser visible, con autenticación manual del usuario y sin evasión de controles.

8. Capa RAW y conservación de evidencia

Antes de extraer datos, la app debe guardar todo artefacto original. Esta capa permite trazabilidad, auditoría, reprocesamiento y explicación posterior de cada movimiento.

```
FinanzasLocal/
├── app.db
├── staging.db
├── logs.db
├── config/
│   ├── accounts.yaml
│   ├── categories.yaml
│   ├── payee_rules.yaml
│   └── connectors.yaml
├── raw/
│   ├── email/
│   ├── mercado_pago/
│   ├── csv/
│   ├── ofx/
│   ├── qif/
│   ├── pdf/
│   ├── images/
│   └── manual/
├── parsed/
├── exports/
│   ├── mmex_csv/
│   ├── mmex_qif/
│   └── audit_reports/
├── backups/
└── workers/
```

Campo de evidencia	Propósito
artifact_id	Identificador único del artefacto.
source_type	email, api, csv, pdf, image, manual.
sha256	Evita reprocesar archivos idénticos y permite auditoría.
storage_path	Ruta local del archivo original.
parser_status	pending, parsed, error, needs_review.

parser_version	Permite saber con qué versión se extrajo el dato.
metadata_json	Datos de correo, API, página PDF, fila CSV u otros metadatos.

9. Modelo de datos local

La base local no debe copiar MMEX uno a uno. Debe ser más rica, porque tiene que conservar evidencia cruda, múltiples fuentes, confianza, hashes, estado de revisión y trazabilidad.

9.1. Tabla source_artifact

```
CREATE TABLE source_artifact (
  artifact_id TEXT PRIMARY KEY,
  source_type TEXT NOT NULL,
  source_name TEXT NOT NULL,
  original_name TEXT,
  storage_path TEXT NOT NULL,
  sha256 TEXT NOT NULL UNIQUE,
  content_type TEXT,
  received_at TEXT,
  imported_at TEXT DEFAULT CURRENT_TIMESTAMP,
  parser_status TEXT DEFAULT 'pending',
  parser_error TEXT,
  metadata_json TEXT
);
```

9.2. Tabla parsed_event

```
CREATE TABLE parsed_event (
  parsed_event_id TEXT PRIMARY KEY,
  artifact_id TEXT NOT NULL,
  parser_name TEXT NOT NULL,
  parser_version TEXT NOT NULL,
  raw_text TEXT,
  extracted_json TEXT NOT NULL,
  confidence REAL DEFAULT 0,
  status TEXT DEFAULT 'parsed',
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (artifact_id) REFERENCES source_artifact(artifact_id)
);
```

9.3. Tabla canonical_tx

```
CREATE TABLE canonical_tx (
  tx_uid TEXT PRIMARY KEY,
  owner TEXT NOT NULL,          -- ricardo | laura | joint
  source_type TEXT NOT NULL,    -- email | api | csv | ofx | qif | pdf | image | manual
  source_name TEXT NOT NULL,    -- gmail | mercado_pago | banco_estado | falabella | mach | bci

  event_date TEXT,
  booking_date TEXT,
  posted_date TEXT,

  amount REAL NOT NULL,
  currency TEXT NOT NULL DEFAULT 'CLP',
  direction TEXT NOT NULL,      -- debit | credit

  account_alias TEXT,
  account_number_masked TEXT,
  card_last4 TEXT,

  merchant_raw TEXT,
  merchant_norm TEXT,

  tx_type TEXT NOT NULL,        -- purchase | bill_payment | transfer_in | transfer_out | internal_transfer |
  refund | fee | rejected_payment
);
```

```

reference_raw TEXT,
reference_norm TEXT,

category_guess TEXT,
subcategory_guess TEXT,
payee_guess TEXT,

tags_json TEXT,
raw_text TEXT,
evidence_json TEXT,

hash_artifact TEXT,
hash_business_strong TEXT,
hash_business_medium TEXT,
fitid_synthetic TEXT,

confidence REAL DEFAULT 0,
reconcile_status TEXT DEFAULT 'new',

mmex_account_id INTEGER,
mmex_tx_id INTEGER,

created_at TEXT DEFAULT CURRENT_TIMESTAMP,
updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

```

9.4. Tablas complementarias

Tabla	Uso
tx_evidence	Relaciona cada transacción con correos, PDFs, filas CSV, JSON API u OCR.
payee_rule	Reglas locales para normalizar comercios y categorías.
account_alias	Mapea tarjetas/cuentas reales a cuentas de MMEX.
import_batch	Registra cada corrida de ingesta, validación y exportación.
review_task	Cola de casos dudosos o errores que requieren decisión humana.
subscription_candidate	Agrupar pagos recurrentes detectados.
export_log	Audita qué movimientos se exportaron, cuándo y hacia qué archivo.

10. Normalización, categorización y reglas

La normalización transforma datos caóticos de bancos y correos en registros homogéneos. Debe operar antes de cualquier deduplicación y antes de cualquier exportación a MMEX.

10.1. Fechas

Campo	Uso
event_date	Fecha real del evento, por ejemplo la compra.
booking_date	Fecha en que el banco contabiliza el movimiento.
posted_date	Fecha que se exportará a MMEX.

Regla

Todas las fechas deben convertirse internamente a ISO YYYY-MM-DD. Nunca exportar fechas ambiguas al libro final.

10.2. Montos

- Guardar amount como valor absoluto.
- Guardar direction como debit o credit.
- Guardar currency como CLP, USD u otra moneda.
- Eliminar símbolos monetarios, separadores regionales y texto residual.
- Aplicar signo solo al exportar, no dentro del modelo canónico.

10.3. Comercios y beneficiarios

Entrada cruda:
PAYPAL *UBER EATS SANTIAGO
UBER* TRIP
UBER EATS CL

Salida normalizada:
Payee: Uber Eats
Categoria: Alimentacion
Subcategoria: Delivery
Tags: delivery, comida

10.4. Estrategia de reglas

Nivel	Mecanismo	Ejemplo
1	Regla exacta	merchant_raw == "NETFLIX".
2	Contains	merchant_raw contiene "UBER EATS".
3	Regex	Patrones bancarios o referencias variables.
4	Fuzzy matching	Variantes de comercio con similitud alta.
5	Historial humano	Aprende de correcciones previas.
6	LLM local opcional	Solo para plantillas ambiguas o nuevas.
7	Revisión manual	Cuando la confianza no alcanza umbral.

```
- name: Uber Eats
  priority: 10
  match:
    contains:
      - "UBER EATS"
      - "PAYPAL *UBER"
  output:
    payee: "Uber Eats"
    category: "Alimentacion"
    subcategory: "Delivery"
    tags:
      - "delivery"
```

11. Dedupe y conciliación

La deduplicación es el componente crítico. El sistema debe permitir reprocesar correos, cartolas y PDFs sin duplicar gastos en MMEX.

11.1. Niveles de deduplicación

Nivel	Técnica	Uso
1	Hash exacto del artefacto	Evita reprocesar el mismo correo, PDF, CSV o JSON API.
2	FITID sintético fuerte	Fecha + monto + moneda + cuenta/tarjeta + referencia.
3	FITID sintético medio	Fecha bucket + monto + moneda + tarjeta + comercio normalizado.
4	Matching difuso	Merchant/memo similar, monto igual, misma tarjeta, ventana de 1 a 3 días.
5	Sliding window amplio	Correo de autorización versus cargo liquidado hasta 14 días después.

FITID fuerte:
sha256(owner + amount + currency + normalized_reference + account_or_card + date_bucket)

FITID medio:
sha256(owner + amount + currency + merchant_norm + card_last4 + date_bucket)

11.2. Precedencia entre fuentes

Dato	Fuente que manda
Monto final	Banco/API/estado de cuenta.
Fecha contable	Banco/API/estado de cuenta.
Comercio legible	Correo, si describe mejor el comercio.
Referencia bancaria	Banco/API.
Categoría	Reglas locales + revisión humana.
OCR	Solo rellena huecos; no derrota fuentes estructuradas.

Regla de seguridad

Si dos fuentes discrepan en monto o dirección, no se fusionan automáticamente. Van a revisión.

11.3. Conciliación de casos críticos

Caso	Tratamiento correcto
Compra con tarjeta	Correo + estado de cuenta se fusionan en una sola canonical_tx con evidencia múltiple.
Pago de tarjeta	Si la tarjeta existe en MMEX, registrar como transferencia interna, no como gasto.
Transferencia Ricardo ↔ Laura	Si ambas cuentas están modeladas, fusionar salida y entrada como internal_transfer.
Pago rechazado	Registrar como evento de alerta; no afecta balance.
Reembolso	Crear transacción separada asociable, no sobrescribir compra original.
Suscripción	Detectar recurrencia por comercio, monto similar, periodicidad y cuenta.

12. Integración con Money Manager Ex

La integración debe ser progresiva. Primero, exportar archivos que MMEX ya sabe importar. Después, cuando el flujo esté probado, evaluar escritura directa sobre SQLite.

12.1. Fase 1: CSV importable

- Generar un CSV por cuenta y por lote.
- Usar el Universal CSV Importer de MMEX.
- Mantener el hash FITID sintético en Notes o campo equivalente durante MVP.
- Marcar movimientos dudosos como Followup o dejarlos fuera hasta revisión.
- No tocar directamente el archivo .mmb en esta etapa.

```
Date,Payee,Amount,Transaction Type,Category,Subcategory,Notes,Number,Status
2026-05-02,Uber Eats,12990,Withdrawal,Alimentacion,Delivery,"Gmail+CSV hash:abc123",,Followup
```

12.2. Fase 2: QIF/OFX generado

- Generar QIF como formato puente si MMEX importa con menor fricción que CSV.
- Evaluar OFX si se desea mayor semántica financiera.
- Mantener FITID sintético para evitar duplicados entre lotes.

12.3. Fase 3: SQLite directo

- Cerrar MMEX antes de escribir.
- Crear backup automático del .mmb/.emb o copia de trabajo.
- Validar esquema con PRAGMA table_info.
- Insertar dentro de BEGIN/COMMIT/ROLLBACK.
- Gestionar PAYEE_V1, CATEGORY_V1, CHECKINGACCOUNT_V1, TAG_V1, TAGLINK_V1 y CUSTOMFIELDDATA_V1.
- Verificar que no existan duplicados por FITID sintético antes de insertar.
- Registrar mmex_tx_id de vuelta en canonical_tx.

12.4. Custom Fields recomendados en MMEX

Campo MMEX	Contenido
source_type	email, csv, pdf, api, manual.
source_name	gmail, mercado_pago, cmr, banco_estado, mach, bci.
fitid_synthetic	Hash único de negocio para deduplicación.
source_ref	Referencia bancaria o ID API.
card_last4	Últimos cuatro dígitos de tarjeta.
owner	Ricardo, Laura o conjunto.
parser_version	Versión del parser que generó la transacción.
artifact_hash	Hash SHA-256 de la evidencia original.

13. Seguridad, privacidad y backups

Elemento	Política recomendada
Tokens OAuth	Guardar cifrados mediante DPAPI o Credential Manager.
Contraseñas bancarias	No guardarlas. La app no debe almacenar credenciales bancarias.
Password .emb	No guardar por defecto; pedir al usuario cuando sea necesario.
Logs	Nunca registrar RUT completo, tarjeta completa, tokens, passwords o PDF completo.
Backups	Crear antes de exportar, antes de escribir en MMEX y diariamente.
Permisos	Principio de mínimo privilegio: solo carpetas configuradas.
IA	Solo local si procesa datos bancarios. No enviar PDFs/correos a modelos cloud.

13.1. Backups sugeridos

```
backups/mmex/mmex_2026-05-02_1530_pre_export.mmb
backups/staging/staging_2026-05-02_1530.db
backups/config/config_2026-05-02_1530.zip
backups/reports/export_audit_2026-05-02_1530.json
```

14. Dashboard financiero

La app debe entregar inteligencia operativa antes de MMEX. El dashboard debe permitir ver problemas y tendencias sin abrir directamente el libro contable.

Panel	Contenido
Resumen mensual	Ingresos, egresos, saldo neto y gasto promedio diario.
Categorías	Gasto por categoría, subcategoría y variación mensual.
Suscripciones	Pagos recurrentes detectados, monto esperado y próxima fecha.
Pagos rechazados	Alertas que no afectan balance, pero requieren acción.
Cuentas por pagar	Deudas, cuotas, vencimientos y tarjetas.
Pendientes de revisión	Movimientos con baja confianza o conflicto de conciliación.
Exportación MMEX	Últimos lotes exportados, estado y errores.
Calidad de datos	Duplicados evitados, parsers fallidos y fuentes sin actualizar.

15. Scheduler y automatización local

Job	Frecuencia sugerida	Observación
Sincronizar Gmail	Cada 15-30 minutos	Solo descarga metadata y correos nuevos.
Leer carpeta de entrada	Cada 5 minutos	Detecta nuevos CSV, PDF, OFX, QIF e imágenes.
Procesar PDFs pendientes	Manual o nocturno	OCR puede ser costoso.
Sincronizar Mercado Pago	Cada 1-3 horas	Respetar límites de API.
Detectar suscripciones	Diario	Analizar recurrencia por comercio/monto.

Crear backup	Diario y antes de exportar	Backup inmutable con timestamp.
Exportar MMEX	Manual en MVP	El usuario aprueba antes de exportar.
Limpieza de logs	Semanal	Mantener auditoría sin exceso de datos sensibles.

Regla de concurrencia

Solo un job puede escribir en staging.db al mismo tiempo. Solo un job puede exportar o escribir hacia MMEX al mismo tiempo.

16. Estructura del proyecto

```
FinanzasLocal.sln
├── FinanzasLocal.Desktop
│   ├── Views/
│   ├── ViewModels/
│   └── App.axaml / App.xaml
├── FinanzasLocal.Core
│   ├── Entities/
│   ├── ValueObjects/
│   ├── Services/
│   ├── Rules/
│   └── Interfaces/
├── FinanzasLocal.Infrastructure
│   ├── SQLite/
│   ├── Repositories/
│   ├── Migrations/
│   ├── Security/
│   └── FileStorage/
├── FinanzasLocal.Connectors
│   ├── Gmail/
│   ├── MercadoPago/
│   ├── FileDrop/
│   └── ManualEntry/
├── FinanzasLocal.MMEX
│   ├── CsvExporter/
│   ├── QifExporter/
│   ├── SqliteWriter/
│   └── SchemaInspector/
├── FinanzasLocal.WorkerHost
│   ├── JobRunner/
│   ├── Scheduler/
│   └── PythonBridge/
└── FinanzasLocal.Tests
    ├── Unit/
    ├── Integration/
    └── GoldenFiles/
```

16.1. Workers Python

```
workers/python/finetl/
├── adapters/
│   ├── email_gmail.py
│   ├── csv_bancoestado.py
│   ├── csv_falabella.py
│   ├── pdf_cmr.py
│   ├── mercado_pago.py
│   └── manual.py
├── extractors/
│   ├── html_text.py
│   ├── regex_bank.py
│   ├── pdf_native.py
│   ├── ocr_image.py
│   └── llm_local.py
├── normalizers/
│   ├── dates.py
│   ├── amounts.py
│   ├── merchants.py
│   └── accounts.py
├── output/
│   └── candidate_json.py
└── tests/
```

17. Contratos JSON entre C# y Python

Para reducir locks y complejidad, Python no debe escribir inicialmente en SQLite. Python recibe un artefacto, lo procesa y devuelve JSON; C# valida y persiste.

```
{
  "parser_name": "gmail_banco_falabella",
  "parser_version": "1.0.0",
  "source_type": "email",
  "source_name": "banco_falabella",
  "artifact_sha256": "abc123",
  "candidates": [
    {
      "event_date": "2026-05-02",
      "amount": 12990,
      "currency": "CLP",
      "direction": "debit",
      "merchant_raw": "UBER EATS SANTIAGO",
      "merchant_norm": "Uber Eats",
      "card_last4": "1234",
      "tx_type": "purchase",
      "reference_raw": null,
      "confidence": 0.92,
      "evidence_excerpt": "Compra realizada por $12.990..."
    }
  ]
}
```

18. Validaciones obligatorias

Nivel	Validaciones
Transacción	Fecha válida, monto > 0, moneda válida, dirección válida, fuente identificada, cuenta mapeada o revisión, hash generado.
Lote	Conteo de artefactos, conteo de candidatos, duplicados detectados, errores, tasa de confianza, balance si existe saldo inicial/final.
Estado de cuenta	saldo_final = saldo_inicial + ingresos - egresos. Si no cuadra, el lote va a revisión.
Antes de exportar	Sin duplicados aprobados, sin cuentas desconocidas, sin fechas inválidas, backup creado, categorías válidas.
Antes de SQLite directo	MMEX cerrado, backup creado, esquema validado, transacción ACID activa, rollback disponible.

19. Roadmap de desarrollo

Fase	Duración sugerida	Entregables
0. Descubrimiento	1 semana	Inventario de cuentas MMEX, bancos, fuentes, categorías, ejemplos reales anonimizados y diseño final de staging.
1. Núcleo local	1-2 semanas	App mínima, SQLite, tablas base, logs, backups y carga manual.
2. Gmail	2 semanas	OAuth, descarga de correos, guardado local, parser HTML/texto y candidatos.
3. Archivos CSV/PDF	2-3 semanas	Watch folder, adaptadores CSV, PDF nativo, OCR y cola de revisión.
4. Dedupe/conciliación	2 semanas	Hashes, fuzzy matching, sliding window, transferencias internas y merge multi-fuente.
5. Exportación MMEX	1-2 semanas	CSV/QIF por cuenta, reporte de exportación y estados exported/imported.
6. Dashboard	2 semanas	Gastos, ingresos, suscripciones, pagos rechazados, cuentas por pagar y calidad de datos.
7. Mercado Pago API	1-2 semanas	OAuth, movimientos, pagos, reembolsos, comisiones y conciliación.
8. Browser asistido	2-3 semanas	Asistente visible para descargar cartolas, sin evasión de controles.
9. SQLite directo MMEX	3-4 semanas	Schema inspector, writer ACID, custom fields, tags, rollback y auditoría.

20. Estrategia de pruebas

- Crear golden files con correos, CSV, PDFs y JSON API reales anonimizados.
- Probar parseo de fechas, montos CLP, comercios, dirección y cuentas.
- Probar deduplicación correo + CSV, correo + PDF y API + correo.
- Probar transferencias internas y pagos de tarjeta para evitar doble gasto.
- Probar pagos rechazados para asegurar que no afecten el balance.
- Probar exportación CSV importable en MMEX.
- Probar reimportación del mismo lote sin duplicados.
- Probar backups y rollback.

```
tests/fixtures/  
├── gmail_bancoestado_compra.eml  
├── gmail_falabella_rechazo.eml  
├── mercadopago_payment.json  
├── cmr_estado_cuenta.pdf  
├── bancoestado_movimientos.csv  
├── mach_notificacion.eml  
└── expected_canonical_tx.json
```

21. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Cambio de plantilla de correo	Parser deja de extraer datos.	Fallback con LLM local, reglas versionadas y cola de revisión.
Duplicados multi-fuente	Balances inflados.	FITID sintético, fuzzy matching y revisión conservadora.
PDF mal extraído	Movimientos incompletos o incorrectos.	Validación de balance, OCR solo como respaldo y revisión humana.
MMEX bloqueado	Fallo de exportación o escritura.	CSV en MVP; locks y chequeo de proceso en fase SQLite.
Corrupción .mmb	Pérdida de libro contable.	Backups automáticos, copia de trabajo, transacciones ACID y rollback.
Token OAuth filtrado	Acceso indebido al correo/API.	DPAPI/Credential Manager, scopes mínimos y rotación.
Sincronización familiar conflictiva	database is locked o versiones divergentes.	Un solo PC fuente de verdad al inicio; Syncthing solo cuando estable.
Scraping bancario inestable	Bloqueos y fallas frecuentes.	No usar como base; preferir archivos y browser asistido visible.

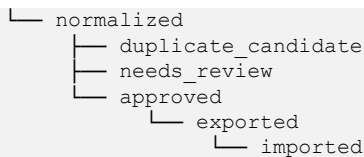
22. Criterios de éxito del MVP

1. Lee correos financieros desde Gmail.
2. Permite cargar CSV/PDF manualmente.
3. Guarda evidencia cruda con hash.
4. Normaliza fecha, monto, cuenta, comercio y tipo.
5. Deduplica correo + estado de cuenta.
6. Detecta pagos rechazados sin afectar balance.
7. Detecta posibles suscripciones.
8. Permite revisión humana de casos dudosos.
9. Exporta CSV compatible con MMEX.
10. Crea backups automáticos.
11. Permite reprocesar lotes sin duplicar.
12. No escribe directamente al .mmb en la primera versión.

23. Apéndices técnicos

23.1. Estados de una transacción

```
raw  
└── parsed
```



Estado	Significado
raw	Artefacto recibido, no procesado.
parsed	Campos preliminares extraídos.
normalized	Fechas, montos, cuenta y comercio estandarizados.
duplicate_candidate	Posible duplicado.
merged	Fusionado con otra fuente.
needs_review	Requiere decisión humana.
approved	Puede exportarse.
exported	Fue enviado a CSV/QIF/MMEX writer.
imported	Se confirmó en MMEX.
ignored	No era transacción útil.
error	Falló parsing o validación.

23.2. Lo que no se debe hacer en la primera versión

No hacer	Razón
Escribir directo al .mmb desde el día uno	Riesgo de corrupción contable.
Depender solo de Gmail	No todos los movimientos llegan por correo.
Automatizar bancos en modo invisible	Inestable, riesgoso y propenso a bloqueo.
Guardar contraseñas bancarias	Riesgo alto e innecesario.
Mezclar todo en Notes de MMEX	Ensucia la contabilidad y dificulta auditoría.
No guardar evidencia	Impide explicar de dónde salió cada movimiento.
No tener revisión humana	Los bancos cambian formatos y los parsers fallan.
Usar IA en nube con PDFs bancarios	Contradice privacidad y soberanía local.

23.3. Arquitectura final recomendada

Síntesis final

Una app desktop local en .NET, con SQLite como staging canónico, workers Python para extracción/OCR/parsing, conectores controlados para Gmail, Mercado Pago y archivos, motor de deduplicación con FITID sintético, cola de revisión humana y exportación segura hacia MMEX por CSV/QIF primero, dejando la escritura directa al .mmb como fase avanzada.

23.4. Documentos base considerados

- informe de claude.pdf
- informe de gemini.pdf
- informe de notebook 1m.pdf
- informe de opneai.pdf